Class Script: Build Your Portfolio Website with AI (2 hours)

How to read this script

- **SAY:** words to speak. Adapt freely; it's written the way you'd talk.
- **DO:** what to do on screen.
- **PROMPT:** what to type into Antigravity. Improvise the wording, but keep the details in **bold**.
- **ASK:** a question for the class, with the answer you're looking for.
- **PPT:** when to switch to the slide deck and exactly what to click, in order.
- ⏱ marks the target time. If you're behind, see "Timing safety valves" at the end.

Students should arrive with Node.js, Antigravity, GitHub and Vercel accounts ready, and their portrait saved as `portrait.png`.

The deck

Open `docs/pptx-build-package/AI-Portfolio-Class-Concepts-ClickPerBeat-v3.pptx` in a second window (or on a projector second screen) before class starts, full-screen it (F5), and leave it parked on slide 1. **Every click plays exactly one small reveal and then stops** — nothing auto-advances, nothing auto-plays through. You click, something appears on screen, you say the line next to it, you click again. That pacing is the whole point: the deck waits for you, not the other way round.

The 10 topics below map 1:1 to the 10 build blocks. Each topic's click table gives you the exact number of clicks, what lands on screen at each one, and a short line to say. Treat the "SAY" column as a starting point, not a script to read verbatim — say it the way you'd actually talk, the same as everywhere else in this document.

⏱ 0:00 · Opening (8 minutes)

DO: Open your live portfolio on the projector. Don't say anything yet. Scroll slowly through the whole site, top to bottom.

SAY:

> "Good morning, everyone. Take a look at this website. It has animations, a projects section, a contact form, and it works on a phone.
>
> How long do you think it would take to build something like this from scratch? A week? A month? Most freshers would say a month, if they know React well.
>
> Today, in the next two hours, every one of you is going to build your own version of this. Your name, your projects, your photo. And at the end of the class, it will be live on the internet with a link you can put on your resume and LinkedIn.
>
> And here's the interesting part: we are not going to type code. Not one line."

ASK: "So if we're not writing code, who is?" → *The AI*.

SAY:

> "Right. But not ChatGPT in a chat window. We're going to use an AI **agent**: an AI that can actually create files, run commands and fix its own errors on our computer. The tool is called Google Antigravity.
>
> I've spent years training developers, and the pattern is always the same: the students who get hired aren't the ones who memorised the most syntax — they're the ones who learned to direct a tool like this and verify what it gives back. That's the actual skill today is built around.

> Now, here's how today works. I'll give the agent an instruction, in plain English. The agent takes a few minutes to build. And instead of us sitting and staring at the screen, I'll use that time — on the slides next to me — to teach you how this actually works: what an agent is, what prompting really means, what MCP is, what a harness is. These are the words every company is talking about right now. By the end, you won't just have a website; you'll understand the technology that built it.
>
> One rule: follow along with me, but don't race ahead. If your agent finishes before mine, read what it wrote. Reading AI output is a skill too.
>
> Everyone, open Antigravity. Create a new folder on your desktop called `my-portfolio`, and open that folder in Antigravity. Hands up when you're there."

DO: Wait for most hands. Help anyone stuck.

⏱ 0:08 · Block 1: Create the project (10 minutes)

SAY:

> "First, every website needs a foundation, like a building needs a structure before we paint the walls. We'll ask the agent to set up a React project. React is what most modern websites use, and companies hire for it. Watch how I write this."

PROMPT:

> Create a new **React project using Vite** in this folder. Add **Tailwind CSS version 4** using the Vite plugin, **Framer Motion** for animations, and **react-icons** for icons. Then start the development server and tell me the

> local link. Also **set up git and make a first commit**, and from now on **commit after every step that works**.

SAY:

> "Send it. Now watch the screen. Don't look away, this is the important part."

Teach while it builds: What is an AI agent?

SAY:

> "Look at what it's doing. It just ran a command. It's reading the output. Now it's deciding what to do next. Let's understand this.
>
> When you use ChatGPT, you ask a question and it gives you an answer. That's it. If it gives you code, *you* have to copy it, *you* have to create the file, *you* have to run it, and when there's an error, *you* go back and paste the error. ChatGPT is a brain in a box. It can only talk.
>
> An agent is a brain **with hands**. Let me show you exactly what that loop looks like."

PPT: Topic 1 — "What is an AI Agent?" (6 clicks)

Click	What appears	SAY
1	Sealed speech-bubble icon	"ChatGPT: a brain in a box. It can only talk."
2	Agent icon; hands pull in a doc and keys	"An agent has hands. It grabs the files, the terminal — it can act on its own."

Click	What appears	SAY
3	Brain badge lands at top of a circle	"**Think.** It plans the next step. 'I need to install Tailwind.'"
4	Arc draws; ACT badge lands right	"**Act.** It uses a tool. It runs `npm install`."
5	Another arc draws; OBSERVE badge lands bottom	"**Observe.** It reads what happened. Success? Error?"
6	Final arc closes; REPEAT badge lands, dot orbits forever	"**Repeat.** It goes back to think. And that circling dot never stops — that's literally what's running on your own screen right now."

ASK: "Suppose the install fails with an error. What will the agent do?" → *It reads the error and tries to fix it.* "Exactly. Same loop. Observe the error, think about the fix, act again. That's why it can fix its own mistakes and ChatGPT can't."

SAY:

> "And this is why companies are excited. A chatbot answers questions. An agent completes tasks. That difference is going to change a lot of jobs, including ours. The people who know how to direct agents will be very valuable. That's what you're learning today."

DO: When the agent finishes, open the local link (usually `http://localhost:5173`).

SAY:

> "There it is. A running React app. Everyone, open your link. Hands up if you see the starter page."

If someone is stuck:

- `npm` **is not recognised:** Node.js isn't installed. Pair them with a neighbour for today.
- **Agent stuck for more than 3 minutes:** stop it and type: "Run npm install, then npm run dev, and tell me the link."

⏱ 0:18 · Block 2: Theme, navbar and footer (12 minutes)

SAY:

> "Now we have a blank building. Before we add rooms, let's decide the paint. Every professional website has a fixed set of colours and fonts. We call it a design system. If you don't fix it at the start, the AI will pick random colours for every section, and your site will look like five different websites stitched together.
>
> Our theme: black background, white text, and one red accent colour. Watch how specific I'm going to be."

PROMPT:

> I'm building my personal portfolio website. Set up the theme: background **black #000000**, text **white**, accent colour **red #ff2a2a**. Use **only these colours** across the whole site. Use the **Inter** font from Google Fonts.
>
> Create these sections in this order, **each in its own component file** with its own id: **Hero, About, Skills, Projects, Experience, Contact**. For now, just a heading in each.
>
> Add a **fixed navbar**: my name on the left, links to every section in the middle, and a red **Hire Me** button on the right. On mobile, show a

> **hamburger menu**. Add a **footer** with my name, social icons and the copyright year. Links should **scroll smoothly** to each section.

Teach while it builds: What makes a good prompt?

SAY:

> "While it's building, let's dissect what I just wrote. This is the most important skill of the day. The quality of what the AI builds depends almost entirely on the quality of your instruction."

PPT: Topic 2 — "Anatomy of a Good Prompt" (5 clicks)

Click	What appears	SAY
1	Title + blank card outline	"A real prompt is a brief, not a wish. It has four stamps."
2	GOAL stamp thuds onto top-left	"**Goal.** 'I'm building my personal portfolio.' Now the AI knows what world we're in."
3	SPECIFICS colour swatch drops into top-right	"**Specifics.** I didn't say 'red'. I said this exact hex code. A hundred reds exist — a hex code is exactly one."
4	CONSTRAINTS fence stamp slams bottom-left	"**Constraints.** 'Use ONLY these colours.' Fences. Without them, the AI wanders."
5	CHECK stamp lands bottom-right, card holds complete	"**Check.** A way to know it's actually done. Skip any one of these four stamps, and the AI fills the blank itself — with its own guess, not yours."

ASK: "What if I had just typed: 'make me a nice portfolio website'? What would the AI have to guess?" → *Colours, sections, fonts, layout, everything*.

SAY:

> "Everything. And here's the thing: the AI will never say 'I don't know what you want.' It will confidently guess. And your guess and its guess will be different. Then you'll spend an hour saying 'no, not like that.'
>
> So remember this line: **the AI is not a mind reader. Every detail you leave out, it guesses.** A good prompt is just a good brief, the same thing a manager gives an employee. Goal, specifics, constraints."

DO: When done, open the site and click each navbar link. Narrow the browser window to show the hamburger menu.

SAY:

> "Black background, red accents, the menu scrolls to each section, and look: on a narrow screen, the hamburger appears. Everyone check yours."

If colours come out wrong: "Use only #000000, #ffffff and #ff2a2a. Replace every other colour in the project."

⏱ 0:30 · Block 3: Your data file (12 minutes)

SAY:

> "Now, the most important block of the day. Our site is empty. It needs *your* details: your name, your projects, your skills.
>
> We could tell the AI to write your name into the Hero, then your skills into Skills, then your projects into Projects. But then your details are scattered

> across ten files. Next month, when you change your phone number, you'll be hunting through all of them.
>
> So instead, we'll put **all** your information in **one** file. Every section reads from that file. Change it once, and the whole site updates. Professionals call this a 'single source of truth'.
>
> And instead of typing everything, we'll make the AI interview us."

PROMPT:

> Create one file, `src/data/portfolioData.js`, that holds **all the text** on my site: my name, role, a short intro, about me, skills grouped by category, projects (title, description, tech stack, live link, GitHub link), experience (role, company, dates, two points each), email, phone, city and social links.
>
> **Every component must read from this file.** No text typed directly into components.
>
> **Interview me** to fill it in. Ask me a few questions at a time. **Do not invent anything.** If I skip something, leave a TODO.

SAY:

> "Now it's going to ask you questions. Answer with your real details. Your real projects, even college projects. Your real internship, if you have one. If you don't have experience yet, that's fine: tell it your training here or your degree.
>
> This is exactly the discipline we drill into every batch here: real projects, committed to GitHub, before a single mock interview. Not tutorials you can't explain under pressure."

DO: Answer the agent's questions live with your own details. This is engaging; read questions out loud.

Teach while students answer: Context and hallucination

PPT: Topic 3 — "Context & Hallucination" (6 clicks) — run this once most students are typing, so the clicks land while they're still answering.

Click	What appears	SAY
1	Context box appears	"Everything inside this box is what the AI actually knows."
2	NAME + ROLE icon flies in	"Your real name, your real role."
3	REAL PROJECT icon flies in	"A real project — not invented."
4	TRAINING icon flies in	"Your real training or degree."
5	Ghost icon appears outside the box, dim, glitching	"Now watch what happens the moment a fact is missing."
6	Ghost solidifies red, labels HALLUCINATED	"It doesn't say 'I don't know.' It invents something and holds it like it's true. 'Built an e-commerce platform serving 10,000 users.' Confident, believable, and completely false. That's **hallucination**."

ASK: "Would you put a fake internship on your resume?" → *No*.

SAY:

> "Then don't let the AI put one on your website. An interviewer will ask you about every project on your portfolio. That's why we said 'do not invent

> anything' and let it interview us. We gave it the context, so it doesn't need to guess."

DO: When done, open `portfolioData.js` on screen and scroll through it.

SAY:

> "Open your `portfolioData.js`. Read it. Is everything true? No 'John Doe', no made-up projects? If something's wrong, just tell the agent: 'change my role to Java Full Stack Developer'. One line."

⏱ 0:42 · Block 4: Hero and About (15 minutes)

SAY:

> "Now the first thing anyone sees on your site: the Hero section. A recruiter spends about six seconds on a portfolio before deciding to scroll or leave. The hero has to say who you are, what you do, and look good while doing it.
>
> Copy your `portrait.png` into the `src/assets` folder now. Tell me when it's there."

PROMPT:

> Build the **Hero** section: full screen, a greeting "Hi, I'm (name)", my **role in red**, a two-line intro, and two buttons, **"View My Work"** (scrolls to Projects) and **"Contact Me"** (scrolls to Contact). Show my portrait from `src/assets/portrait.png` on the right.
>
> The hero must **fit in one screen on laptop and on phone**. On mobile, text first and the image below it.

> Then build the **About** section: my portrait in a card on one side, my about text and **three quick stats** on the other (like projects built, technologies, years of learning).
>
> Read all text from **portfolioData.js**.

Teach while it builds: The harness

SAY:

> "Here's a question. Antigravity, Cursor, GitHub Copilot, Claude Code: they all use AI models. Some even use the *same* model. So why do they behave so differently?
>
> The answer is the **harness**. And I mean that literally — watch what that word actually means."

PPT: Topic 4 — "The Harness" (10 clicks)

Click	What appears	SAY
1	Title + an empty cart fades in, no horse yet	"A harness is literally a set of straps connecting raw pulling power to the thing it operates. Right now, nothing's harnessed to anything."
2	The horse trots in, straps snap taut; MODEL badge pops in with a power-up burst	"The **model** — the horse. Gemini, Claude, GPT. Same raw power, every time, no matter which app it's strapped into."
3	Current flows down the wire; TOOLS toolbox badge straps onto the cart	"**Tools** — the kit strapped on. Creating files, running the terminal, opening a browser."

Click	What appears	SAY
4	Wire glows toward the horse's head; CONTEXT lantern badge settles in	"**Context** — the lantern. Lighting up only what it's allowed to see."
5	Wire glows toward the driver's bench; PERMISSIONS reins badge settles in	"**Permissions** — the reins. Someone's always holding them, ready to pull back."
6	Fully harnessed horse and cart hold, then fade out	"One horse, fully harnessed to one cart. That's Antigravity."
7	Kicker text: SAME HORSE. DIFFERENT HARNESS.	*(let it read itself for a beat)*
8	A light racing cart flashes in — same glowing horse, labeled ANTIGRAVITY	"Same horse. A completely different harness."
9	A sturdy farm cart flashes in, labeled CURSOR	"Same horse — Cursor."
10	An ornate royal carriage flashes in, labeled CLAUDE CODE; all three hold	"Same horse — Claude Code. Three different harnesses. One constant underneath: the model."

ASK: "Why do you think Antigravity sometimes stops and asks your permission before running a command?" → *Safety. So it doesn't delete things or run something dangerous without a human approving.*

SAY:

> "Exactly. The harness keeps a human in control. A good engineer doesn't just pick the best model; they pick the right harness and set it up well. That's a

> real job now, sometimes called 'AI engineering'."

DO: When done, show the hero. Then narrow the browser to phone width (or open the browser's phone view with your on-screen device toggle).

SAY:

> "Look at that. Your name, your role in red, your portrait. Check it at phone width too. Does everything fit? Nothing overlapping?"

⏱ 0:57 · Checkpoint: Fix something live (5 minutes)

SAY:

> "Real talk: the AI is never perfect on the first try. Something on your screen probably looks a bit off. Mine too. So let's learn the most useful skill of all: how to ask for a fix."

PPT: Topic 5 — "How to Ask for a Fix" (5 clicks) — run this before you touch your own site, so students see the formula once, clean, before applying it themselves.

Click	What appears	SAY
1	Phone mockup fades in, overlap bug already visible	"Here's a real bug. The portrait is sitting on top of the Save button."
2	Magnifying glass scales in, finds the overlap	"**Where.** Point at it exactly — Hero section, on mobile."

Click	What appears	SAY
3	Warning badge appears, pulses over the overlap	"**What's wrong.** Say it plainly — it overlaps the buttons."
4	The fix: portrait physically slides down and shrinks	"**What you want.** Watch it resolve, live — below the buttons, narrower."
5	Dashed fence draws around the rest of the screen, locking it	"**The fence.** 'Don't change anything else.' That line is what stops the agent from fixing ten things you liked."

DO: Pick something imperfect on your own screen. If everything looks fine, ask for a deliberate change (for example, a bigger portrait on mobile).

PROMPT (adapt to your real issue):

> In the **Hero section, on mobile**, the portrait **overlaps the buttons**. Move the portrait **below the buttons** and limit its width to **70% of the screen**. **Don't change anything else.**

SAY:

> "Same four parts you just watched. And even better than words: take a screenshot and paste it into the chat. A picture removes all guessing.
>
> Your turn. Find one thing on your site you'd improve, and fix it using these four parts. Two minutes."

⏱ 1:02 · Block 5: Skills and Projects (13 minutes)

SAY:

> "Next, the two sections recruiters actually read: your skills and your projects. Projects especially. This is your proof of work."

PROMPT:

> Build the **Skills** section: group skills into cards by category (like Frontend, Backend, Tools). Show each skill with its **icon from react-icons** and a **red hover effect**.
>
> Build the **Projects** section: one card per project with the title, short description, tech stack as small tags, and **"Live"** and **"Code"** buttons that open in a **new tab**. Use a **grid on laptop** and **one column on phone**.
>
> Read everything from **portfolioData.js**.

Teach while it builds: MCP (Model Context Protocol)

SAY:

> "Let me ask you something. How many different chargers did you have five years ago? Phone, laptop, earphones, all different. Now? Mostly one: USB-C. One standard plug for everything.
>
> AI had the same problem."

PPT: Topic 6 — "MCP: USB-C for AI" (5 clicks)

Click	What appears	SAY
1	Title fades in; MCP hub badge pops in	"Every AI app wanted to connect to tools: GitHub, Drive, Figma, databases. Every

Click	What appears	SAY
	and pulses	connection built by hand — a mess."
2	GitHub spoke draws in, node + label appear, dot starts travelling	"So in late 2024, Anthropic — the company behind Claude — released a standard called **MCP**. A **GitHub MCP** lets the agent create repos and review code for you."
3	Figma spoke draws in	"A **Figma MCP** — a designer makes a design, the agent reads it directly and builds it."
4	Database spoke draws in	"A **database MCP** — you ask in English, 'how many users signed up this week?', and the agent queries the database for you."
5	Drive spoke draws in; all four dots keep travelling	"One plug. Current literally flowing to all of them at once. In about fifteen minutes, with a GitHub MCP connected, the agent could even create our repository for us."

ASK: "If you were building a tool for a company, why would you want it to support MCP?" → *So every AI app can use it without custom work.*

SAY:

> "Exactly. Build it once, works everywhere. That's why it spread so fast — it's now used by OpenAI, Google, Microsoft, almost everyone."

DO: When done, click a project's buttons and show the one-column layout on phone width.

⏱ 1:15 · Block 6: Experience and Contact (12 minutes)

SAY:

> "Two more sections. Your experience, and a way for recruiters to contact you, which is the whole point of a portfolio."

PROMPT:

> Build the **Experience** section as a **vertical timeline**: role, company, dates and two points each, with a **red line and dots**.
>
> Build the **Contact** section: a form with **name, email and message**. Set it up for **EmailJS**, reading the keys from a `.env.local` file with variables that start with `VITE_`. **If the keys are missing, the form should open the visitor's email app instead** with the message filled in. Add `.env.local` **to** `.gitignore`.

Teach while it builds: Agentic AI, rules files and secrets

SAY:

> "Three quick ideas. Notice your role today — you're not the coder, you're the manager."

PPT: Topic 7 — "Agentic AI, Rules & Secrets" (6 clicks)

Click	What appears	SAY
1	Manager icon appears	"**Agentic AI.** You set the goal."
2	Crossfades to robot icon	"It completes the task on its own. That's the difference from a chatbot — and where the industry is heading."
3	Book appears and opens	"**Rules files.** We've repeated 'use only our colours' many times — you can save that permanently. Antigravity has rules, Claude Code uses `CLAUDE.md`. The agent reads it on every single task, like a company handbook."
4	Key appears	"**Secrets.** The contact form needs keys from EmailJS. A key is a password for a service."
5	Padlock snaps shut over the key	"We lock it in `.env.local`, and tell git to ignore that file."
6	"Never commit keys" warning badge appears	"Bots scan GitHub for leaked keys and find them within minutes. Never paste a real key into a chat, and never commit it."

ASK: "Why does it matter if a key ends up on GitHub?" → *Anyone can see it and use your account.*

SAY:

> "Right. People have woken up to huge cloud bills from exactly this."

DO: When done, show the timeline, then submit the contact form to show the email app opening.

SAY:

> "Today your form opens the email app. That works. Setting up EmailJS so it sends directly is part of your homework; the playbook walks you through it."

⏱ 1:27 · Block 7: Animations and mobile polish (10 minutes)

SAY:

> "Our site works. Now let's make it feel alive, and make sure it's solid. Most recruiters will open your link on their phone, not a laptop."

PROMPT:

> Add gentle **fade-in and slide-up animations** as each section scrolls into view, using Framer Motion. Keep them **subtle**, and **turn them off for users who prefer reduced motion**.
>
> Then check **every section at phone width (375px)** and fix anything that **overlaps, overflows, or is too small to tap**.
>
> Finally, run `npm run lint` and `npm run build` and fix every error.

Teach while it builds: Verification

SAY:

> "Here's something nobody tells you about AI. It can be **confidently**

> **wrong**."

PPT: Topic 8 — "Verify Everything" (6 clicks)

Click	What appears	SAY
1	Robot badge appears, claims "DONE"	"It will say 'Done! Everything works perfectly!'"
2	Red WRONG stamp slams down over the claim	"...when a button is actually broken. It's not lying — it just can't always check everything. So who checks? You do."
3	CLICK checklist icon pops in, green checkmark	"Click every button."
4	TEST (phone) icon pops in, green checkmark	"Test it on your actual phone."
5	ERRORS (bug search) icon pops in, green checkmark	"Read the error messages."
6	BUILD (wrench) icon pops in, green checkmark; final state holds	"Run `lint` and `build`. Every one of these is on you now — that's the real shift in this industry."

SAY:

> " `lint` checks code quality, like a grammar checker for code. `build` prepares the site for publishing. If `build` fails on your laptop, it will fail on the internet too. So we always run it before going live.
>
> A pro tip: ask the agent to 'test your own work and tell me anything you couldn't verify.' A good agent will be honest about what it didn't check.

> And remember the git commits we've been making after every step? That's your **undo button**. If the agent ever breaks something badly, one command takes you back to the last working version. That's why we committed from the very first step."

DO: Scroll your site to show the animations. Show `npm run build` succeeding in the terminal.

⏱ 1:37 · Block 8: Go live (13 minutes)

SAY:

> "This is the moment. Right now, your website only exists on your laptop. Let's put it on the internet. Two steps: GitHub stores your code, and Vercel publishes it."

PROMPT:

> Create a **README.md** with the project title, a placeholder for the live link, and the tech stack. **Commit everything.** Then help me **push this to a new GitHub repository** named **(my-name)-portfolio**, and walk me through any login steps.

SAY:

> "If a GitHub login window pops up, that's normal the first time. Sign in."

Teach while it pushes: Git and CI/CD

SAY:

> "**Git** is the save history of your code: every commit is a save point. **GitHub** is the cloud copy of that history, so your code is safe even if your laptop dies, and recruiters can see it."

PPT: Topic 9 — "Git & CI/CD" (5 clicks)

Click	What appears	SAY
1	CODE node appears	"Your code, on your laptop."
2	Line draws to COMMIT node	"A commit — a save point."
3	Line draws to GITHUB node	"Pushed to GitHub — the cloud copy."
4	Line draws to LIVE node, which glows	"And live, on the internet."
5	Loop-back arc draws in; dot travels CODE→LIVE while a second dot loops LIVE→CODE	"Watch that dot keep moving. Every time you push new code, this whole chain replays itself automatically. That's **CI/CD** — continuous integration and continuous deployment."

DO: On screen:

1. Go to **vercel.com → Add New → Project**.
2. Import the GitHub repo.
3. Leave the defaults; Vercel detects Vite automatically.
4. Click **Deploy**.

SAY (while it deploys):

> "Nobody uploads files by hand. That's how real companies ship software, sometimes hundreds of times a day."

DO: Open the live link. Then open it on your phone and hold it up.

SAY:

> "It's live. Everyone, deploy yours. When it's done, **open your link on your own phone**."

DO: Give them a full minute. This is the high point of the class. Let them enjoy it.

DO (if time allows, a 1-minute demo): Change one word in `portfolioData.js`, commit and push, and show Vercel redeploying automatically.

If a build fails: open the Vercel build log, copy the error, and give it to the agent: "The Vercel build failed with this error: (paste). Fix it."

⏱ 1:50 · Wrap-up (10 minutes)

SAY:

> "Two hours ago, you had an empty folder. Now you have a live portfolio website, on the internet, with your name on it. And none of you typed code.
>
> But more important than the website: let's recap what you learned."

PPT: Topic 10 — "What You Actually Learned" (6 clicks) — ask each question, let students answer, **then** click to light the badge and confirm.

Click	What appears	ASK first, then confirm
1	Dim grid appears; Agent Loop badge lights gold	"What is an agent?" → *An AI with tools, working in a loop: think, act, observe, repeat.*
2	Good Prompt badge lights gold	"What makes a good prompt?" → *Goal, specifics, constraints. The AI isn't a mind reader.*
3	Context & Hallucination badge lights gold	"What's a context window? What's hallucination?" → *The AI's short-term memory. When it lacks facts, it invents them.*
4	The Harness badge lights gold	"What's a harness?" → *Everything around the model: tools, context, permissions. The model is the engine; the harness is the car.*
5	MCP badge lights gold	"What's MCP?" → *USB-C for AI: one standard way to connect AI to tools.*
6	Verify Everything badge lights gold; full grid holds	"Who checks the AI's work?" → *We do. Verify everything, commit often.*

SAY:

> "Six ideas. You didn't just build a website today — you now know how every AI tool out there actually works.
>
> Your homework, to take this from good to great. It's all in the playbook, step by step:
>
> 1. Add the **ID badge that flips into a visiting card** with a QR code, like on my site.

> 2. Make your **talking intro video** and put it in the hero.
> 3. Set up **EmailJS** so your contact form sends real emails.
> 4. Add the **preloader**, the **'My Process'** section and the **fluid cursor effect**.
> 5. Submit your site to **Google Search Console**, write your **LinkedIn post**, and upload your **YouTube video**.
>
> Put your live link on your resume and your LinkedIn today. Send me your link; I want to see every one of them.
>
> This is exactly the kind of project you'd walk into a mock interview with here and actually be able to defend, line by line — because you built every part of it with intent, not by copying a tutorial. That's the whole difference.
>
> Any questions?"

DO: Take questions, including anything you parked on the board during class.

Backup plans

Problem	What to do
A student's agent fails or freezes	Stop it, re-send a smaller instruction. If still stuck, pair with a neighbour.
Antigravity usage limit hit	Pair with a neighbour; continue at home.
Wi-Fi too slow for installs	Pair students (two per laptop) for Block 1.
Site breaks after a change	"Go back to the last commit" is one instruction to the agent.

Problem	What to do
You're running behind	See the safety valves below.
A student finishes early	Ask them to help neighbours, or start homework item 1.
PPT won't advance / wrong clip plays	Each click = exactly one beat. If you click too fast and skip one, press ← to go back one step, don't restart the slide.

Timing safety valves

If you're behind, cut in this order:

1. **Block 7 animations:** keep only the phone check and `npm run build`.
2. **Experience timeline:** fold it into About as a short list.
3. **Skill icons:** plain text tags are fine.
4. **PPT click tables:** every topic's clicks can be thinned — skip the slowest-landing clicks (e.g. Harness's three mini-vehicle reveals can collapse to just showing the first one) and talk over the rest from memory.

Never cut: the data file (Block 3), the live fix (checkpoint), and going live (Block 8). Those are what students remember.

Engagement reminders

- **Before each prompt:** ask "what do you think it'll build?" Then compare.
- **Once in the class:** send a vague prompt on purpose, show the weak result, then fix it with a specific one.
- **When the agent runs a command:** point and say "act, observe: that's the loop".
- **First three students done in each block:** they help their neighbours.

- **Off-topic questions:** write them on the board and answer them during Q&A.
- **Switching to the PPT:** glance at it, click, say the line, glance back at the agent. Don't read the table out loud — it's your cue card, not the students' subtitle.